

UNIVERSIDAD NACIONAL DE SAN AGUSTÍN

Facultad de Ingeniería de Producción y Servicios

Escuela Profesional de Ciencia de la Computación



Laboratorio:
**Análisis Empírico de Complejidad del Problema
del Máximo Subarreglo**

Trabajo Realizado por:
Huaylla Guzman Roid Niro

Grupo: A

AREQUIPA - PERÚ
2026

1. Repositorio del Proyecto

El código fuente completo del proyecto, con las tres implementaciones del algoritmo de máximo subarreglo y el script de pruebas, se encuentra disponible en: <https://github.com/rhuayllag/maximoSubArreglo>

2. Descripción del Problema

El problema del **máximo subarreglo** consiste en, dado un arreglo de números enteros (positivos y negativos), encontrar la suma máxima que se puede obtener a partir de un subarreglo contiguo. En este laboratorio se implementaron y compararon tres algoritmos con distinta complejidad:

- **Fuerza bruta cúbica**, $O(n^3)$: prueba todos los subarreglos posibles y recalcula la suma de cada uno desde cero.
- **Fuerza bruta cuadrática**, $O(n^2)$: prueba todos los subarreglos posibles pero acumula la suma incrementalmente, evitando recalcularla.
- **Algoritmo de Kadane**, $O(n)$: recorre el arreglo una sola vez, decidiendo en cada posición si conviene extender el subarreglo actual o comenzar uno nuevo.

3. Implementaciones

3.1. Versión Cúbica $O(n^3)$

```
1 def max_subarray_cubica(a):
2     n = len(a)
3     mejor = float('-inf')
4
5     for i in range(n):
6         for j in range(i, n):
7             suma = 0
8             for k in range(i, j + 1):
9                 suma += a[k]
10            if suma > mejor:
11                mejor = suma
12
13     return mejor
```

Listing 1: Fuerza bruta cúbica

Por cada par de índices (i, j) se recorre nuevamente el subarreglo para sumar sus elementos, lo que da tres ciclos anidados y complejidad $O(n^3)$.

3.2. Versión Cuadrática $O(n^2)$

```
1 def max_subarray_cuadratica(a):
```

```
1  n = len(a)
2  mejor = float('-inf')
3
4
5  for i in range(n):
6      suma = 0
7      for j in range(i, n):
8          suma += a[j]
9          if suma > mejor:
10             mejor = suma
11
12  return mejor
```

Listing 2: Fuerza bruta cuadrática

Aquí se elimina el tercer ciclo: la suma se acumula incrementalmente al extender el subarreglo hacia la derecha, reduciendo la complejidad a $O(n^2)$.

3.3. Algoritmo de Kadane $O(n)$

```
1  def max_subarray_kadane(a):
2      aqui = a[0]
3      global_max = a[0]
4
5      for i in range(1, len(a)):
6          aqui = max(a[i], aqui + a[i])
7          global_max = max(global_max, aqui)
8
9      return global_max
```

Listing 3: Algoritmo de Kadane

En cada posición se decide, con programación dinámica, si conviene continuar el subarreglo que termina en el elemento anterior o comenzar uno nuevo desde el elemento actual (**aqui**). Se recorre el arreglo una sola vez: complejidad $O(n)$.

3.4. Script de Comparación (main.py)

El script `main.py` importa las tres funciones, valida que produzcan el mismo resultado en casos borde (arreglo con todos positivos, todos negativos, un solo elemento, etc.) y mide el tiempo de ejecución de cada una para distintos tamaños de entrada con `time.perf_counter()`.

```
1  def medir_tiempo(func, a):
2      inicio = time.perf_counter()
3      resultado = func(a)
4      fin = time.perf_counter()
5      return resultado, fin - inicio
```

Listing 4: Medición de tiempos

4. Entorno de Medición

Ítem	Detalle
Lenguaje	Python 3.x
Sistema operativo	Windows
Procesador	Por completar
Memoria RAM	Por completar

Cuadro 1: Entorno donde se realizaron las mediciones

5. Resultados: Tiempos Medidos

Los siguientes tiempos se obtuvieron ejecutando `main.py` con arreglos aleatorios de enteros en $[-100, 100]$.

n	Cúbica $O(n^3)$	Cuadrática $O(n^2)$	Kadane $O(n)$
1000	5.777233 s	0.024904 s	0.000100 s
2000	47.104218 s	0.096780 s	0.000197 s
4000	388.447199 s	0.385860 s	0.000397 s
8000	≈ 3185 s (≈ 53 min)*	≈ 1.52 s*	≈ 0.0008 s*

Cuadro 2: Tiempos de ejecución por tamaño de entrada. *Para $n = 8000$, el valor de la versión cúbica no fue medido (tomaría cerca de una hora); los tres valores son una estimación (ver Sección 6).

En todos los casos, incluidos los de borde (arreglo con solo positivos, solo negativos, un único elemento, ceros), los tres algoritmos devolvieron exactamente el mismo resultado, confirmando que las tres implementaciones son correctas.

6. Predicción para $n = 8000$

Con los tres tiempos medidos ($n = 1000, 2000, 4000$) se ajustó un modelo de la forma $t(n) = c \cdot n^k$, calculando el exponente empírico como:

$$k = \frac{\log(t(4000)/t(1000))}{\log(4000/1000)}$$

Algoritmo	Exponente empírico k	Exponente teórico	Predicción $n = 8000$
Cúbica	3.04	3	3185 s (≈ 53 min)
Cuadrática	1.98	2	1.52 s
Kadane	0.995	1	0.0008 s

Cuadro 3: Exponente empírico vs. teórico y predicción para $n = 8000$

Comparación predicho vs. medido: el exponente calculado a partir de las mediciones coincide, en los tres casos, con la complejidad teórica del algoritmo (3, 2 y 1 respectivamente), con un error menor al 4 %. Esto valida el modelo y permite usarlo con confianza para extrapolar a $n = 8000$ y a $n = 10^8$ sin necesidad de ejecutar realmente la versión cúbica en esos tamaños.

7. Estimación para $n = 10^8$

Usando el mismo modelo, con la constante c calculada a partir del tiempo medido en $n = 4000$:

Algoritmo	Tiempo estimado en $n = 10^8$	¿Termina en menos de 1 min?
Cúbica $O(n^3)$	$\approx 8,7 \times 10^{15}$ s (≈ 276 millones de años)	No
Cuadrática $O(n^2)$	$\approx 1,9 \times 10^8$ s (≈ 6 años)	No
Kadane $O(n)$	$\approx 9,4$ s	Sí

Cuadro 4: Estimación de tiempo para $n = 10^8$

Solo **Kadane** logra procesar un arreglo de 100 millones de elementos en menos de un minuto. El tiempo por elemento medido ($\approx 9,9 \times 10^{-8}$ s/elemento) se mantiene prácticamente constante entre $n = 1000$ y $n = 4000$, por lo que escalarlo linealmente hasta 10^8 es razonable. En cambio, el crecimiento no lineal (n^2 y n^3) hace que cuadrática y cúbica se vuelvan completamente imprácticas mucho antes de llegar a 10^8 .

8. Conclusiones

1. **Correctitud:** en todas las pruebas (desde $n = 5$ hasta $n = 4000$) los tres algoritmos devolvieron exactamente el mismo resultado, confirmando que las implementaciones de fuerza bruta (cuadrática y cúbica) son equivalentes a Kadane, solo que con distinta eficiencia.
2. **La complejidad teórica se confirma empíricamente:** al duplicar n , el tiempo de la cúbica se multiplica por ≈ 8 (2^3), el de la cuadrática por ≈ 4 (2^2) y el de Kadane por ≈ 2 (2^1), exactamente lo esperado para $O(n^3)$, $O(n^2)$ y $O(n)$.
3. **La diferencia se vuelve crítica al escalar:** para n pequeño (decenas o cientos) cualquier algoritmo sirve, pero a partir de unos pocos miles de elementos la fuerza bruta cúbica ya es impracticable, y para $n = 10^8$ solo Kadane es viable en la práctica.
4. **Recomendación:** para el problema de máximo subarreglo, Kadane ($O(n)$) es la única opción razonable a partir de tamaños moderados de entrada; las versiones cuadrática y cúbica solo tienen valor didáctico para ilustrar la mejora de complejidad algorítmica.